Chronic Kidney Disease prediction

CLASSIFICATION

Basic information about dataset

Total data	399 rows × 28 columns
Dependent	classification_yes 1 249 0 150

Data Pre-processing

1. One-hot encoding to convert categorial values into numerical values
2. Standardization

Best model

Both Logistic Regression and SVM are giving best results.
F1 Macro value = 0.9924

Jupyter 1.Classification_LR-Grid_CKD_Casestudy Last Checkpoint: 26 minutes ago

File Edit View Run Kernel Settings Help

Code

```
f1_macro = f1_score(y_test, grid_predictions, average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)
```

```
The f1_macro value for best parameter {'penalty': 'l2', 'solver': 'sag'}: 0.9924946382275899
```

[15]:
```
print(cm)
```

```
[[51  0]
 [ 1 81]]
```

[16]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

[17]:
```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

[17]: 1.0

jupyter 1.Classification_LR-Grid_CKD_Casestudy Last Checkpoint: 26 minutes ago

File Edit View Run Kernel Settings Help

Code

```
f1_macro = f1_score(y_test, grid_predictions, average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)
```

```
The f1_macro value for best parameter {'penalty': 'l2', 'solver': 'sag'}: 0.9924946382275899
```

[15]:
```
print(cm)
```

```
[[51  0]
 [ 1 81]]
```

[16]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

[17]:
```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

[17]: 1.0